

Movie Review Project

The **Movie Review Project** is a full-stack web application that allows users to browse and review movies. Guests can view top movies and read reviews, while registered users can submit reviews, rate movies, and manage their preferences. Admins have access to additional functionalities for managing content.

Tech Stack Used

- **Backend:** Node.js, Express.js
- **Frontend:** EJS (Embedded JavaScript Templates), HTML, CSS, Tailwind
- **Database:** MySQL
- **Middleware:** Express middleware for authentication and user status handling
- **Authentication:** JSON Web Tokens (JWT), bcryptjs and Cookies

Project Structure:

#Root Directory

- .env - Stores environment variables like database credentials
- .gitignore - Specifies files to ignore in version control
- package.json - Project dependencies and scripts
- server.js - Main entry point for the application

#Configuration

- config/db.js - Establishes connection to the MySQL database

#Controllers (Handles business logic)

- controllers/authController.js - Manages authentication (login, logout, register)
- controllers/adminController.js - Admin-related functionalities
- controllers/reviewController.js - Manages movie reviews

#Middleware

- middleware/authMiddleware.js - Middleware for checking user status and authentication

#Public Directory (Frontend static files)

- public/css/style.css - Main stylesheet
- public/images/ - Stores image assets
- public/js/ - Frontend JavaScript files:
 - home.js - Handles homepage interactions
 - guestNav.js, navbar.js, register.js, login.js - Handles UI interactions
 - categories.js, dashboard.js, review.js - Dynamic UI behavior

#Routes (Defines API endpoints)

- routes/authRoutes.js - Authentication routes (login, register, logout)
- routes/viewRoutes.js - Routes for serving pages without authentication (homepage, reviews, categories)
- routes/protectedRoutes.js - Routes that require authentication
- routes/adminRoutes.js - Routes for admin operations

#Service Layer

- service/movieService.js - Handles database operations related to movies

#Views (Frontend templates using EJS)

- views/partials/ - Reusable components (navbar, head, navigation bars)
- views/ - Page templates:
 - home.ejs - Homepage
 - login.ejs, register.ejs - Authentication pages
 - review.ejs - Movie review page
 - dashboard.ejs - User dashboard
 - error.ejs - Error handling page

Installation & Setup

- **Clone the repository:**
 - `git clone <repository_url>`
 - `cd <project_directory>`
- **Install dependencies:**
 - Run `npm install` to install the required dependencies.
- **Create a .env file in the root directory and add the required environment variables:**
 - `PORT=3000`
 - `DB_HOST=your_db_host`
 - `DB_USER=your_db_user`
 - `DB_PASSWORD=your_db_password`
 - `DB_NAME=your_db_name`
- **Start the server:**
 - Run `npm start` to launch the application.
 - The server will run on `http://localhost:3000/` by default.

How the Project Works

1. Guests can browse top movies and read reviews.
2. Users can register, log in, submit reviews, favorite movies, and rate movies.
3. Admins can manage content and user reviews.
4. The application uses middleware to check user status but does not block guest access.
5. The backend handles database interactions and serves the EJS frontend dynamically.

BackEnd JS File Documentation

server.js

This document provides an overview and explanation of the `server.js` file used in the MovieReviews project. This file is the backbone of the application, setting up the Express server and routing various requests to their respective handlers.

File Overview

The `server.js` file initializes an Express application, configures middleware, and defines routes for handling API and web requests. Additionally, it manages error handling and sets the application to listen on a specified port.

Key Components

Imports

The file begins by importing necessary modules:

```
const express = require('express');
```

The `express` module is the core of the application.

```
require('dotenv').config();
```

This line loads environment variables from a `.env` file into `process.env`.

```
const db = require('./config/db');
```

This imports the database configuration, ready to connect to the database.

```
const cookieParser = require('cookie-parser');
```

`cookie-parser` middleware is used for parsing cookies attached to the client request objects.

Additionally, various route files are imported:

```
const authRoutes = require('./routes/authRoutes');
```

```
const viewRoutes = require('./routes/viewRoutes');
```

```
const protectedRoutes = require('./routes/protectedRoutes');
```

```
const adminRoutes = require('./routes/adminRoutes');
```

These routes manage different aspects of the application, including authentication, views, protected resources, and admin functionalities.

Setting up the Server

Next, we create an instance of the Express application:

```
const app = express();
```

The port on which the server listens is determined:

```
const PORT = process.env.PORT || 3000;
```

Here, it checks the `PORT` environmental variable or defaults to `3000`.

Middleware Configuration

The server is configured to use various middleware:

```
app.set('view engine', 'ejs');
```

This sets the view engine to EJS (Embedded JavaScript templating), enabling render support for HTML views.

```
app.use(express.static('public'));
```

This allows serving static files such as images, CSS files, and JavaScript files from the `public` directory.

```
app.use(express.json());
```

Enables Express to parse incoming JSON requests.

```
app.use(express.urlencoded({ extended: true }));
```

This allows Express to parse URL-encoded data, which is commonly used for form submissions.

```
app.use(cookieParser());
```

This enables cookie parsing, allowing you to manage cookies in the application easily.

Route Handling

Next, we set up the routes to handle incoming requests:

```
app.use('/api/auth', authRoutes);
```

This attaches the `authRoutes` to the endpoint `/api/auth`.

```
app.use('/', viewRoutes);
```

This attaches the `viewRoutes` to the root endpoint.

```
app.use('/', protectedRoutes);
```

This attaches the `protectedRoutes` also to the root, allowing access to protected resources.

```
app.use('/admin', adminRoutes);
```

This attaches the `adminRoutes` to the endpoint `/admin`.

Error Handling

We have two middleware functions for handling errors and 404 not found errors:

```
app.use((req, res) => {  
  res.status(404).render('error', {  
    status: 404,  
    message: 'Page not found',  
    title: 'Error',  
  });  
});
```

This function catches requests that do not match any routes and returns a 404 error page.

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(err.status || 500).render('error', {  
    status: err.status || 500,  
    message: err.message || 'Something went wrong',  
    title: 'Error',  
  });  
});
```

This function handles any errors that occur in the app, logging the error and sending a response with the appropriate status.

Starting the Server

Finally, the server is started to listen on the predefined port:

```
app.listen(PORT, () => { console.log(`Server is running on port ${PORT}.`); });
```

This logs a message to the console confirming that the server is running.

Config

db.js

This file establishes a connection pool to a MySQL database using the MySQL2 library. It simplifies database interactions in the MovieReviews project.

Overview

The `db.js` file is responsible for creating a connection pool to a MySQL database. It retrieves database configuration from environment variables, allowing for flexible deployment in various environments without hardcoding sensitive information.

Importing MySQL2 Library

We begin by importing the MySQL2 package that provides a way to interact with the MySQL database.

```
const mysql = require("mysql2");
```

Code Explanation:

- This line imports the `mysql2` module, which is required for creating a connection pool.

Creating a Connection Pool

The connection pool is created using the `mysql.createPool` method which is safe and efficient for handling multiple requests.

```
const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
});
```

Code Explanation:

- **host:** The hostname for the database, typically localhost or an IP address. It's pulled from the environment variable `DB_HOST`.
- **user:** The username for accessing the database, retrieved from `DB_USER`.
- **password:** The password associated with the username, retrieved from `DB_PASS`.
- **database:** The specific database to connect to, identified by `DB_NAME`.
- **waitForConnections:** When set to true, it allows the pool to wait for a connection to be freed up instead of throwing an error.
- **connectionLimit:** This specifies the maximum number of connections to create at once, set to 10 in our case.
- **queueLimit:** The maximum number of connection requests the pool will queue, set to 0 for unlimited requests.

Getting a Connection

Once the connection pool has been set up, the next part of the code attempts to get a connection and handle any errors.

```
pool.getConnection((err, connection) => {
  if (err) {
    console.error("Database connection failed:", err);
  } else {
    console.log("Connected to MySQL database.");
    connection.release();
  }
});
```

Code Explanation:

- The `getConnection` method is called to obtain a connection from the pool.
- If an error occurs during connection, the error message is logged to the console for debugging purposes.
- Upon successful connection, a message confirms the connection, and the connection is released back to the pool for future use.

Exporting the Pool

Finally, we export the pool with promises enabled for better asynchronous handling of database queries.

```
module.exports = pool.promise();
```

Code Explanation:

- This line exports the connection pool, enabling other parts of the application to use it for making database queries asynchronously with promise support.

Routes

authRoutes.js

This document provides a detailed overview of the `authRoutes.js` file found in the MovieReviews project. This file serves as the routing layer for authentication functionalities, handling user registration and login requests.

File Overview

The `authRoutes.js` file defines the routes for user authentication in the MovieReviews application. Using the Express framework, it connects HTTP request methods to specific controller functions that handle the logic for registering and logging in users.

Imports

The file begins by importing the required modules:

```
const express = require("express");
const { register, login } = require("../controllers/authController");
```

Here, we use `require` to include the Express library and the `register` and `login` functions from the `authController.js` file. This sets up the necessary components for defining our routes.

Router Configuration

A router instance is created using `express.Router()` which will be used to define our application's routes:

```
const {router} = express.Router();
```

This `router` instance will handle incoming requests for authentication.

Defining Routes

The authentication routes are defined using `router.post`. This method associates an HTTP POST request to specific endpoints with their handling functions:

User Registration

The route for user registration is defined as follows:

```
router.post("/register", register);
```

When a POST request is made to `/register`, the `register` function from the `authController` is invoked to handle the registration logic.

User Login

Similarly, the route for user login is defined like this:

```
router.post("/login", login);
```

When a POST request goes to `/login`, the `login` function from the `authController` is called to process the login details.

Exporting the Router

Finally, the configured router is exported so it can be utilized in other parts of the application:

```
module.exports = router;
```

This makes it possible to include these routes in the main application file and effectively manage authentication-related requests.

viewRoute.js

This documentation provides a comprehensive overview of the **viewRoutes.js** file, which is part of the MovieReviews project. This file is responsible for handling various routes related to movie reviews, ensuring users can interact with the app efficiently.

File Overview

The **viewRoutes.js** file defines a set of routes using the Express framework to render views and serve data related to movies and user interactions. These routes include accessing movie details, reviewing movies, user login and registration, and searching for movies.

Imports

The file starts by importing necessary modules and components:

- **express**: The Express framework for building web applications.

```
const {express} = require('express');
```

- **router**: An instance of Express's router to define route handlers.

```
const {router} = express.Router();
```

- **logout**: A function imported from the authentication controller to handle user logout.

```
const { logout } = require('../controllers/authController');
```

- **movieService**: Contains functions to interact with movie data.

```
const {
  getTopTenMovies,
  getMovieDetails,
  getTopMoviesPerCategory,
  getAllMovies,
} = require('../service/movieService');
```

- **authMiddleware**: Middleware to handle user status if logged in or not only.

Routes

Home Route

GET /

This route handles the home page. It checks the user's status and retrieves the top ten movies to display.

```
router.get('/', checkUserStatusOnly, async (req, res) => {
  const movies = await getTopTenMovies();
  res.render('home', { movies, title: 'HOME PAGE' });
});
```

Guest Review Route

GET /guestReview/:id

This route retrieves details of a specific movie by its ID for guest reviews. If the movie is not found, it returns a 404 error.

```
router.get('/guestReview/:id', checkUserStatusOnly, async (req, res) => {
  const movieId = req.params.id;
  try {
    const movieDetails = await getMovieDetails(movieId);
    if (!movieDetails) {
      return res.status(404).json({ message: 'Movie not found' });
    }
    const { movie, comments, commenters, comment_times } = movieDetails;
  });
```

Registration Route

GET /register

This route renders the registration page for new users.

```
router.get('/register', (req, res) => { ... });
```

Login Route

GET /login

This route renders the login page for users to log into their accounts.

```
router.get('/login', (req, res) => { ... });
```

Menu Categories Route

GET /menu/categories

This route retrieves the top movies categorized for display, checking if the user is logged in.

```
router.get('/menu/categories', async (req, res) => {
  const token = req.cookies.token
  //logic to check the user status if logged in or not//
  try {
    const filteredMovies = await getTopMoviesPerCategory();
    res.status(200).json({
      filteredMovies,
      userIsLoggedIn: res.locals.userIsLoggedIn,
    });
  });
```


All Movies Route

GET /allMovies

This route retrieves all movies when the user is logged in and displays them. If no movies are found, it returns a 401 error.

```
router.get('/allMovies', checkUserStatusOnly, async (req, res) => {
  try {
    const movies = await getAllMovies();

    if (!movies) {
      return res.status(401).json({ message: 'No Movies Found' });
    }

    res.render('allMovies', { movies, title: 'All Movies' });
  }
});
```

Search Movies Route

GET /search

This route allows users to search for movies based on a query string. It filters movies based on user input.

```
router.get('/search', async (req, res) => {
  try {
    const query = req.query.q.trim().toLowerCase()

    const movies = await getAllMovies()

    const filteredMovies = movies.filter((movie) =>
      movie.title.toLowerCase().includes(query)
    )

    res.json(filteredMovies);
  }
});
```

Logout Route

GET /logout

This route invokes the logout function to successfully log out the user.

```
router.get('/logout', logout);
```

Error Handling Route

GET /error

This route displays an error page based on the status code and message provided in the query parameters.

```
router.get('/error', (req, res) => {
  const status = Number(req.query.status) || 500
  const message = req.query.message || 'Something went wrong'
});
```

Middleware

The **checkUserStatusOnly** middleware is employed in several routes to ensure that certain functionalities will be different based on user status.

```
const { checkUserStatusOnly } = require('../middleware/authMiddleware');
```

Exporting the Router

Finally, the configured router is exported so it can be utilized in other parts of the application:

```
module.exports = router;
```

This makes it possible to include these routes in the main application file and effectively manage authentication-related requests.

protectedRoutes.js

This documentation provides an overview of the **protectedRoutes.js** file in the **MovieReviews Project**. This file defines the API routes that are only accessible to authorized users. It facilitates interaction with movie reviews, favorites, and user-specific data. The routes utilize middleware for authentication and interact with various service and controller functions.

Imports

At the start of the file, we import necessary modules and functions which help in handling routing and serving requests:

- **express** - The Express framework is used for building the web application.

```
const {express} = require('express');
```

- **router** - An instance of an Express router to manage our routes.

```
const {router} = express.Router();
```

- **rate, comment, favorite** - Functions from **reviewController**, handling reviews.

```
const { rate, comment, favorite } = require('../controllers/reviewController');
```

- **verifyToken** - A middleware function for authenticating user requests.

```
const { verifyToken } = require('../middleware/authMiddleware');
```

- **getTopTenMovies, getMovieDetails, getFavoriteMovies** - Functions from **movieService** to access movie data.

```
const {
  getTopTenMovies,
  getMovieDetails,
  getFavoriteMovies,
} = require('../service/movieService');
```

Routes Overview

We define several route endpoints using the router object. Below are the details of each route:

1. GET /home

This route retrieves the top ten movies for the homepage.

Code snippet:

```
router.get('/home', verifyToken, async (req, res) => {
  const movies = await getTopTenMovies();
  res.render('home', { user: req.user, movies, title: 'HOME PAGE' });
});
```

It uses the **verifyToken** middleware to ensure the user is authenticated before fetching movie data and rendering the home page.

2. GET /review/:id

This route retrieves detailed information about a specific movie identified by `:id` and provides the user with an option to add comments or rate it.

Code snippet:

```
router.get('/review/:id', verifyToken, async (req, res) => {
  const movieId = req.params.id;
  const movieDetails = await getMovieDetails(movieId, req.user.id);
  try {
    const movieDetails = await getMovieDetails(movieId, userId);

    if (!movieDetails) {
      return res.redirect("/error?status=404&message=Movie not found");
    }

    const {
      movie,
      comments,
      commenters,
      comment_times,
      userRating,
      userHasFavorited,
    } = movieDetails;
  });
```

If the movie is found, it renders the review page with the movie details; if not found, it sends a 404 status and render error page.

3. GET /favorite

This route fetches the user's favorite movies.

Code snippet:

```
router.get('/favorite', verifyToken, async (req, res) => {
  const favoriteMovies = await getFavoriteMovies(req.user.id);
  try {
    const favoriteMovies = await getFavoriteMovies(userId);

    if (favoriteMovies.length === 0) {
      return res.render('favorite', {
        message: 'No favorite movies added',
        favoriteMovies: [],
        title: 'Favorites',
      });
    }
  });
});
```

It checks if the user has any favorite movies and returns an appropriate message if none are found.

4. POST /movies/:id/rate

This route is for submitting a rating for a specific movie.

Code snippet:

```
router.post('/movies/:id/rate', verifyToken, rate);
```

The **rate** function will handle the logic for recording the rating.

5. POST /movies/:id/comment

This route is for adding comments to a specific movie.

Code snippet:

```
router.post('/movies/:id/comment', verifyToken, comment);
```

The **comment** function will manage the comment submission process.

6. POST /movies/:id/favorite

This route allows users to mark a movie as a favorite.

Code snippet:

```
router.post('/movies/:id/favorite', verifyToken, favorite);
```

The **favorite** function will handle the logic for adding a movie to the user's favorites.

Exporting the Router

Finally, the configured router is exported so it can be utilized in other parts of the application:

```
module.exports = router;
```


adminRoutes.js

Overview

The **adminRoutes.js** file defines a set of routes that provide administrative functionalities for the MovieReviews project. These routes allow admins to perform several tasks such as viewing the dashboard, adding or updating movies, and managing user roles.

Dependencies

This file begins by importing necessary dependencies:

```
const {express} = require('express');
```

express: This is a web framework for Node.js, optimized to create server-side applications.

```
const { verifyToken, isAdmin } = require('../middleware/authMiddleware');
```

verifyToken and **isAdmin:** These middleware functions are used to authenticate requests and ensure that only authorized users can access the admin routes.

```
const { getAllDetailsDashboard, addMovies, updateMovie, deleteMovie, changeUserRole, deleteUser } = require(
```

The functions imported from **adminController** provide the core functionalities required for handling the admin-related operations.

Router Setup

A new router instance is created to define the admin routes:

```
const {router} = express.Router();
```

This router will handle the HTTP requests mapped to various endpoints defined below.

Routes

The file defines several routes, each utilizing the express router. Below is a detailed explanation of each route:

1. Dashboard Route

This GET route retrieves details for the admin dashboard:

```
router.get('/dashboard', verifyToken, isAdmin, async (req, res) => {  
  try {  
    const dashboardDetails = await getAllDetailsDashboard();  
    res.render('dashboard', { dashboardDetails, title: 'Admin Dashboard' });  
  }  
});
```

It uses the **verifyToken** and **isAdmin** middleware to ensure security before fetching data with **getAllDetailsDashboard()**. If successful, it renders the dashboard view with the retrieved details.

Example Usage:

```
GET /admin/dashboard
```

2. Add Movies Route

This POST route allows admins to add new movies:

```
router.post('/addMovies', verifyToken, isAdmin, addMovies);
```

It calls the **addMovies** controller function once the request is verified and authorized.

Example Usage:

```
POST /admin/addMovies
```

3. Update Movie Route

This PATCH route is used to update existing movies:

```
router.patch('/updateMovie/:id', verifyToken, isAdmin, updateMovie);
```

The **:id** parameter is the ID of the movie being updated and is validated through middleware.

Example Usage:

```
PATCH /admin/updateMovie/1
```

4. Delete Movie Route

This DELETE route removes a movie by its ID:

```
router.delete('/deleteMovie/:id', verifyToken, isAdmin, deleteMovie);
```

As with the update route, it utilizes the movie's **:id** parameter.

Example Usage:

```
DELETE /admin/deleteMovie/1
```

5. Change User Role Route

This PATCH route changes the role of a user:

```
router.patch('/userRole/:id', verifyToken, isAdmin, changeUserRole);
```

It accepts a user ID to modify their role as defined in the **changeUserRole** function.

Example Usage:

```
PATCH /admin/userRole/1
```

6. Delete User Route

This DELETE route is used to remove a user:

```
router.delete('/deleteUser/:id', verifyToken, isAdmin, deleteUser);
```

The endpoint allows the removal of a user by their ID.

Example Usage:

```
DELETE /admin/deleteUser/1
```

Exporting the Router

Finally, the router instance is exported for use in other parts of the application:

```
module.exports = router;
```

This statement allows the defined routes to be imported and utilized wherever necessary within the project.

Middleware

authMiddleware.js

This file contains middleware functions for verifying user authentication and authorization in a web application. It utilizes JSON Web Tokens (JWT) to ensure that the user is logged in and has the appropriate permissions to access the resources.

Library Imports

The file begins by importing the `jsonwebtoken` library, which is essential for working with JWTs.

```
const jwt = require('jsonwebtoken');
```

Functions Overview

This file exports three middleware functions:

- **verifyToken**: Validates the token and sets user information if valid.
- **isAdmin**: Checks if the logged-in user has admin privileges.
- **checkUserStatusOnly**: Simply checks the user's login status without enforcing any restrictions.

verifyToken Function

This function verifies that a user is authenticated by checking the presence and validity of their JWT.

```
exports.verifyToken = (req, res, next) => { ... };
```

Here's a step-by-step breakdown of what it does:

1. It retrieves the token from the request cookies:

```
const token = req.cookies.token;
```

2. If no token is found, it redirects the user to an error page:

```
return res.redirect('/error?status=403&message=Access+denied.+Please+log+in');
```

3. If a token is found, it attempts to verify it using a secret stored in the environment variables:

```
const decoded = jwt.verify(token, process.env.JWT_SECRET);
```

4. Upon successful verification, user information is added to the request object, and a flag is set in the response locals:

```
req.user = decoded.user;  
res.locals.userIsLoggedIn = true;  
res.locals.isAdmin = req.user.role === 'admin';
```

5. If the token is invalid or expired, appropriate cookies are cleared, and the user is redirected to an error page:

```
catch (error) {
  console.error('Invalid token:', error);

  if (error.name === 'TokenExpiredError') {
    res.clearCookie('token');
    res.locals.userIsLoggedIn = false;
    return res.redirect('/?expired=true');
  }

  res.clearCookie('token');
  return res.redirect(
    '/error?status=403&message=Invalid+token.+Please+log+in+again'
  );
}
```

This function is typically used in routes where authentication is required.

isAdmin Function

The `isAdmin` function checks if the logged-in user is an administrator. If not, access is denied.

```
exports.isAdmin = (req, res, next) => { ... };
```

How it works:

1. It checks whether the `req.user` object is present and confirms the user's role:

```
if (!req.user || req.user.role !== 'admin') { ... }
```

2. If the user is not an admin, it responds with a 403 status and an access denied message:

```
return res.status(403).json({ message: 'Access denied. Admins only!' });
```

This function is important for protecting admin-level routes from unauthorized access.

checkUserStatusOnly Function

This function serves as a simpler alternative to `verifyToken` in that it checks only whether a user is logged in or not.

```
exports.checkUserStatusOnly = (req, res, next) => { ... };
```

Key steps include:

1. Attempt to retrieve the token similarly to the first function:

```
const token = req.cookies.token;
```

2. If no token is found, it sets a flag indicating the user is not logged in:

```
res.locals.userIsLoggedIn = false;
```

3. If a token is present, it verifies the token and updates the response locals accordingly:

```
res.locals.userIsLoggedIn = true;  
res.locals.isAdmin = req.user.role === 'admin';
```

4. Similar to **verifyToken**, it handles token expiration and invalid cases:

```
catch (error) {  
  console.error('Invalid token:', error);  
  
  if (error.name === 'TokenExpiredError') {  
    res.clearCookie('token');  
    res.locals.userIsLoggedIn = false;  
    return res.redirect('/?expired=true');  
  }  
  
  res.clearCookie('token');  
  return res.redirect(  
    '/error?status=403&message=Invalid+token.+Please+log+in+again'  
  );  
}
```

This is useful for cases where knowledge of user login status is needed without strict access control.

Controllers

authController.js

This document provides a comprehensive overview of the authentication controller used in the MovieReviews project. The authentication controller handles user registration, login, and logout functionalities with robust security measures.

Overview

The `authController.js` file is responsible for managing user authentication processes, including registration, login, and logout. It utilizes `bcrypt` for password hashing and JSON Web Tokens (JWT) for secure session management. The controller connects to a database to validate user information and perform necessary operations.

Imports

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const db = require('../config/db');
```

Functions

register

This function handles user registration by validating input data, checking for existing users, hashing the password, and saving the new user to the database.

```
exports.register = async (req, res) => {
```

It first extracts user information from the `request body`, including `username`, `email`, `password`, and `confirmPassword`. If any fields are missing, it returns a 400 status code with an error message. Also extract the URL the user comes from for redirection.

```
const { username, email, password, confirmPassword } = req.body
const redirectTo = req.body.redirect || '/home'
```

Next, the function checks if the passwords match. If they do not, a 403 status code is returned with an appropriate message.

```
if (password !== confirmPassword) {
  return res.status(403).json({ message: 'Passwords do not match' });
}
```

After passing these validations, the function queries the database to ensure that the entered email does not already exist. If it finds an existing user, it returns a 400 error.

```
const [existingUser] = await db.query(
  'SELECT * FROM users WHERE email = ?',
  [email]
);

if (existingUser.length > 0) {
  return res.status(400).json({ message: 'Email already exists.' });
}
```


Assuming all validations are satisfactory, the password is hashed using bcrypt, and the new user is inserted into the `users` table.

```
const salt = await bcrypt.genSalt(10)
const hashedPassword = await bcrypt.hash(password, salt)

const result = await db.query(
  'INSERT INTO users (name, email, password , role) VALUES (?, ?, ? , ?)',
  [username, email, hashedPassword, 'user']
)
```

A token is then created using JWT, with an expiration time of one hour. If successful, the token is sent back as a secure cookie, and a success message is returned along with a redirect URL.

```
const payload = { user: { id: userId, role: 'user' } };
const token = jwt.sign(payload, process.env.JWT_SECRET, {
  expiresIn: '1h',
});

if (!token)
  return res.status(500).json({ message: 'Error generating token.' });

res.cookie('token', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'Strict',
});

res
  .status(201)
  .json({ message: 'User registered successfully.', redirect: redirectTo });
```

login

The login function allows users to authenticate by entering their email and password and extract the redirect path. It first checks that both fields are filled out; if not, a 400 status code is sent back.

```
exports.login = async (req, res) => {
```

```
  const { email, password } = req.body
  const redirectTo = req.body.redirect || '/home';
  if (!email || !password) {
    return res.status(400).json({ message: 'Please fill in all fields.' });
  }
```

The function retrieves the user from the database using the provided email. If no user is found, it returns a 404 status code indicating that the user does not exist.

```
const [existingUser] = await db.query(
  'SELECT * FROM users WHERE email = ?',
  [email]
);
if (existingUser.length === 0) {
  return res.status(404).json({ message: 'User not found.' });
}
```

If the user is located, bcrypt is used to verify the entered password against the stored hashed password. A mismatch results in a 401 status code for invalid password attempts.

```
const [user] = existingUser;

const isMatch = await bcrypt.compare(password, user.password)
if (!isMatch) {
  return res.status(401).json({ 'message': 'Invalid Password.' })
}
```

Upon successful verification, a token is generated, similar to the registration process, and sent back as a secure cookie. A success message is relayed, including a redirect URL.

```
const payload = { user: { id: user.id, role: user.role } };
const token = jwt.sign(payload, process.env.JWT_SECRET, {
  expiresIn: '1h',
});

if (!token)
  return res.status(500).json({ message: 'Error generating token.' });

res.cookie('token', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'Strict',
});
res.json({ message: 'User logged in successfully.', redirect: redirectTo });
```

logout

The logout function is straightforward but essential. It clears the authentication token from the user's cookies, effectively logging them out. This function then redirects the user to the home page.

```
exports.logout = (req, res) => {
  res.clearCookie('token', {
    httpOnly: true,
    secure: true,
    sameSite: 'Strict',
  });
};
```

reviewController.js

Overview

The `reviewController.js` file primarily serves as the interface for handling user submissions related to movie ratings, comments, and the toggling of favorite statuses. It communicates with the database to store information as needed based on user requests.

Imports

```
const db = require('../config/db');
```

Functions

1. Rate a Movie

The `rate` function allows users to rate a movie between 1 and 5 stars. It performs validation to ensure the movie exists and that the rating is within the acceptable range.

```
exports.rate = async (req, res) => {
  const { rating } = req.body;
  const movieId = req.params.id;
  const userId = req.user.id;

  try {
    const [movie] = await db.query("SELECT * FROM movies WHERE id = ?", [
      movieId,
    ]);
    if (movie.length === 0) {
      return res.status(404).json({ message: "Movie not found." });
    }

    if (!rating || rating < 1 || rating > 5) {
      return res
        .status(400)
        .json({ message: 'Please provide a rating between 1 and 5.' });
    }
  }
```

Also provide functionality to update or insert on the database based on if the user had previous rate or not.

```
const [existingRating] = await db.query(
  'SELECT * FROM ratings WHERE movie_id = ? AND user_id = ?',
  [movieId, userId]
);

if (existingRating.length > 0) {
  await db.query(
    'UPDATE ratings SET rating = ? WHERE movie_id = ? AND user_id = ?',
    [rating, movieId, userId]
  );
} else {
  await db.query(
    'INSERT INTO ratings (rating, movie_id, user_id) VALUES (?, ?, ?)',
    [rating, movieId, userId]
  );
}
```

And then return the average rate of that movie with the new user rate after submitting.

```
const [[updatedRating]] = await db.query(
  'SELECT ROUND(AVG(rating), 1) AS newAverageRating, COUNT(*) AS newNumberOfRaters FROM ratings WHERE movie_id = ? AND user_id = ?',
  [movieId, userId]
);

res.status(201).json({
  message: 'Rating saved successfully.',
  newAverageRating: updatedRating.newAverageRating || 0,
  newNumberOfRaters: updatedRating.newNumberOfRaters || 0,
});
```

Usage Example: Sending a POST request to `/movies/:id/rate` with a rating in the body results in the movie's rating being updated or inserted.

2. Comment on a Movie

The `comment` function allows users to add comments to movies. It checks if the movie exists and ensures the comment is not empty before saving it.

```
exports.comment = async (req, res) => {
  const { comment } = req.body;
  const movieId = req.params.id;
  const userId = req.user.id;

  try {
    const [movie] = await db.query('SELECT * FROM movies WHERE id = ?', [
      movieId,
    ]);

    if (movie.length === 0) {
      return res.status(404).json({ message: 'Movie not found.' });
    }

    if (!comment.trim()) {
      return res.status(400).json({ message: 'Please provide a comment.' });
    }
  }
}
```

And then return the new user comment with the user name and the time of the new comment after adding it to the database.

```
await db.query(
  'INSERT INTO comments (comment, movie_id, user_id) VALUES (?, ?, ?)',
  [comment, movieId, userId]
);

const [[newComment]] = await db.query(
  `SELECT users.name AS commenter, comments.createdAt AS commentTime FROM comments
  JOIN users ON comments.user_id = users.id WHERE comments.movie_id = ? ORDER BY comments.id DESC LIMIT 1`
  [movieId]
);
```

```
res.status(201).json({
  message: 'Comment added successfully.',
  commenter: newComment.commenter,
  commentTime: newComment.commentTime,
  comment: comment,
});
```

Usage Example: Sending a POST request to `/movies/:id/comment` with a comment in the body adds the comment to the specified movie.

3. Toggle Favorite Status

The `favorite` function allows users to add or remove movies from their favorites list. It checks if the movie already exists first.

```
exports.favorite = async (req, res) => {
  const userId = req.user.id;
  const movieId = req.params.id;
  try {
    const [movie] = await db.query('SELECT * FROM movies WHERE id = ?', [
      movieId,
    ]);
    if (movie.length === 0) {
      return res.status(404).json({ message: 'Movie not found.' });
    }
  }
```

And then it checks if the movie is in the user's favorites and acts accordingly, either adding the movie or deleting it.

```
const [existingFavorite] = await db.query(
  'SELECT * FROM favorites WHERE user_id = ? AND movie_id = ?',
  [userId, movieId]
);
if (existingFavorite.length > 0) {
  await db.query(
    'DELETE FROM favorites WHERE user_id = ? AND movie_id = ?',
    [userId, movieId]
  );
  return res.json({ message: 'Removed from favorites', favorited: false });
}
```

```
else {
  await db.query(
    'INSERT INTO favorites (user_id, movie_id) VALUES (?, ?)',
    [userId, movieId]
  );

  return res
    .status(201)
    .json({ message: 'Added to favorites', favorited: true });
}
```

Usage Example: Sending a POST request to `/movies/:id/favorite` adds or removes the movie from the user's favorites based on its current status.

adminController.js

This documentation provides a detailed overview of the `adminController.js` file, which is responsible for managing administrative functions in the Movie Reviews Project. The controller interacts with the database to perform operations such as fetching movie and user details, adding, updating, or deleting movies and users, and changing user roles.

Overview of Admin Functions

1- getAllDetailsDashboard()

This asynchronous function retrieves various statistics intended for the admin dashboard, such as the count of movies, users, comments, and ratings.

Function Logic:

- It queries the `movies`, `users`, and counts of `comments`, `movies`, `users` and `ratings` from the database.
- It returns an object with the counts and the full lists of movies and users.
- If there's an error during the database operations, it catches the error and throws a new error.

```
exports.getAllDetailsDashboard = async (req, res) => {
  try {
    const [movies] = await db.query('SELECT * FROM movies');
    const [users] = await db.query('SELECT * FROM users');
    const [commentCount] = await db.query(
      'SELECT COUNT(*) AS total_comments FROM comments'
    );
    const [ratingCount] = await db.query(
      'SELECT COUNT(*) AS total_ratings FROM ratings'
    );

    return {
      movieCount: movies.length || 0,
      userCount: users.length || 0,
      commentCount: commentCount[0].total_comments || 0,
      ratingCount: ratingCount[0].total_ratings || 0,
      movies: movies || [],
      users: users || [],
    }
  }
}
```

2- addMovies()

This function adds a new movie to the database based on the provided data in the request body.

```
exports.addMovies = async (req, res) => {
```

Function Logic:

- It checks if all required fields (title, description, release year, producer, image URL, category, and duration) are provided.

```
const {
  title,
  description,
  release_year,
  producer,
  image_url,
  category,
  duration,
} = req.body;
if (
  !title ||
  !description ||
  !release_year ||
  !producer ||
  !image_url ||
  !category ||
  !duration
) {
  return res.status(400).json({ message: 'Please provide all fields' });
}
```

- The title is converted to lowercase to ensure uniqueness, and a query checks if the same movie already exists.

```
const lowercaseTitle = title.toLowerCase().trim();

const [existingMovie] = await db.query(
  'SELECT * FROM movies WHERE LOWER(title) = ?',
  [lowercaseTitle]
);

if (existingMovie.length > 0) {
  return res.status(400).json({ message: 'Movie already exists!' });
}
```

- If the movie doesn't exist, it inserts the new movie into the database.

```
const [result] = await db.query(
  `INSERT INTO movies (title, description, release_year, producer, image_url, category, duration)
  VALUES (?, ?, ?, ?, ?, ?, ?)`,
  [
    title,
    description,
    release_year,
    producer,
    image_url,
    category,
    duration,
  ]
);
```

- If the operation is successful, it sends a confirmation response; otherwise, it handles errors accordingly.

```
if (result.affectedRows > 0) {  
  return res.status(201).json({ message: 'Movie added successfully!' });  
} else {  
  return res.status(500).json({ message: 'Failed to add movie' });  
}
```

3- updateMovie()

This function updates an existing movie's details based on the provided ID and data in the request body.

```
exports.updateMovie = async (req, res) => {  
  const { id } = req.params;
```

Function Logic:

- It checks whether at least one field (title, description, etc.) has been provided for the update.

```
const {  
  title,  
  description,  
  release_year,  
  producer,  
  image_url,  
  category,  
  duration,  
} = req.body;  
  
if (  
  !title &&  
  !description &&  
  !release_year &&  
  !producer &&  
  !image_url &&  
  !category &&  
  !duration  
) {  
  return res  
    .status(400)  
    .json({ message: 'Please provide at least one field to update' });  
}
```

- If the movie exists, it builds an SQL update query based on the fields that are provided with new values.

```
let hasChanges = false;
const updateFields = [];
const updateValues = [];
if (title && title.toLowerCase() !== movie[0].title.toLowerCase()) {
  updateFields.push('title = ?');
  updateValues.push(title);
  hasChanges = true;
}
if (
  description &&
  description.toLowerCase() !== movie[0].description.toLowerCase()
) {
  updateFields.push('description = ?');
  updateValues.push(description);
  hasChanges = true;
}
if (release_year && release_year !== movie[0].release_year) {
  updateFields.push('release_year = ?');
  updateValues.push(release_year);
  hasChanges = true;
}
if (
  producer &&
  producer.toLowerCase() !== movie[0].producer.toLowerCase()
) {
  updateFields.push('producer = ?');
  updateValues.push(producer);
  hasChanges = true;
}
if (
  image_url &&
  image_url.toLowerCase() !== movie[0].image_url.toLowerCase()
) {
  updateFields.push('image_url = ?');
  updateValues.push(image_url);
  hasChanges = true;
}
if (
  category &&
  category.toLowerCase() !== movie[0].category.toLowerCase()
) {
  updateFields.push('category = ?');
  updateValues.push(category);
  hasChanges = true;
}
if (duration && duration !== movie[0].duration) {
  updateFields.push('duration = ?');
  updateValues.push(duration);
  hasChanges = true;
}
```


- If there are changes made, it performs the update; otherwise, it sends a message stating that no changes were made.

```
if (!hasChanges) {
  return res
    .status(400)
    .json({ message: 'No changes were made to the movie' });
}

updateValues.push(id);

const [result] = await db.query(
  `UPDATE movies SET ${updateFields.join(', ')} WHERE id = ?`,
  updateValues
);

if (result.affectedRows > 0) {
  return res.status(200).json({ message: 'Movie updated successfully' });
} else {
  return res.status(500).json({ message: 'Failed to update movie' });
}
```

4- deleteMovie()

This function deletes a movie from the database based on its ID provided in the request parameters.

```
exports.deleteMovie = async (req, res) => {
  const { id } = req.params;
```

Function Logic:

- It checks if the movie with the given ID exists.

```
try {
  const [movie] = await db.query('SELECT * FROM movies WHERE id = ?', [id]);

  if (movie.length === 0) {
    return res.status(404).json({ message: 'Movie not found' });
  }
}
```

- If it exists, it executes a delete operation.

```
const [result] = await db.query('DELETE FROM movies WHERE id = ?', [id]);
```

- It sends a confirmation message if the deletion is successful or responds with an error message if not.

```
if (result.affectedRows > 0) {
  return res.status(200).json({ message: 'Movie deleted successfully' });
} else {
  return res.status(500).json({ message: 'Failed to delete movie' });
}
```

5- changeUserRole()

This function updates a user's role (e.g., user to admin) based on the user ID and the new role sent in the request.

```
exports.changeUserRole = async (req, res) => {
  const id = parseInt(req.params.id, 10);
  const role = req.body.role.toLowerCase();
```

Function Logic:

- It checks that the provided role is either 'user' or 'admin' also the ID is a number.

```
try {
  if (!['user', 'admin'].includes(role.toLowerCase())) {
    return res
      .status(400)
      .json({ message: 'Invalid role. Allowed values: user, admin' });
  }

  if (isNaN(id)) {
    return res.status(400).json({ message: 'Invalid user ID' });
  }
```

- It ensures that the requesting admin can only change user roles if they are not trying to change another admin's role.

```
const loggedInAdminRole = req.user.role;

const [user] = await db.query('SELECT role FROM users WHERE id = ?', [id]);

if (user.length === 0) {
  return res.status(404).json({ message: 'User not found' });
}

const targetUserRole = user[0].role;

if (targetUserRole === 'admin' && loggedInAdminRole === 'admin') {
  return res
    .status(403)
    .json({ message: 'You cannot change another admin's role' });
}
```

- If valid, it performs the role update in the database and sends an appropriate response.

```
if (targetUserRole === role) {
  return res.status(400).json({ message: 'User already has this role' });
}

const [result] = await db.query('UPDATE users SET role = ? WHERE id = ?', [
  role,
  id,
]);
```

```

if (result.affectedRows > 0) {
  return res.json({ message: 'User role updated successfully!' });
} else {
  return res.status(500).json({ message: 'Failed to update user role' });
}

```

6- deleteUser()

This function deletes a user from the database based on the user ID provided in the request parameters.

```

exports.deleteUser = async (req, res) => {
  const id = parseInt(req.params.id, 10);
  const loggedInAdminRole = req.user.role;

```

Function Logic:

- It first checks that the ID is a number then checks if the user exists. If not found, it sends a 404 response.

```

try {
  if (isNaN(id)) {
    return res.status(400).json({ message: 'Invalid user ID' });
  }

  const [user] = await db.query('SELECT role FROM users WHERE id = ?', [id]);

  if (user.length === 0) {
    return res.status(404).json({ message: 'User not found' });
  }

```

- It ensures that admins cannot delete other admins.

```

const targetUserRole = user[0].role;

if (targetUserRole === 'admin' && loggedInAdminRole === 'admin') {
  return res
    .status(403)
    .json({ message: 'Admins cannot delete other admins' });
}

```

- If the user is valid, it deletes the user from the database and sends a confirmation message.

```

const [result] = await db.query('DELETE FROM users WHERE id = ?', [id]);

if (result.affectedRows > 0) {
  return res.json({ message: 'User deleted successfully!' });
} else {
  return res.status(500).json({ message: 'Failed to delete user' });
}

```

Service

movieService.js

File Overview

The `movieService.js` file is responsible for handling all operations related to movies. It connects to the database, executes queries, and processes results to give the necessary movie data to other parts of the application. The main functions in this file enable fetching the top ten movies, detailed movie information, favorite movies for a user, and other related tasks.

Functions

getTopTenMovies

This asynchronous function retrieves the top ten movies based on their average ratings. It makes a query to the database to fetch the specified number of movies and their related rating and comment statistics.

Parameters

- **limit** (number): *Optional*. The maximum number of movies to return. Defaults to 10.

Returns

An array of movie objects, each containing movie details along with the number of raters, average rating, and comment count.

Code

```
async function getTopTenMovies(limit = 10) {
  try {
    const [movies] = await db.query(
      `SELECT
        m.*,
        COALESCE(r.rating_count, 0) AS number_of_raters,
        COALESCE(r.average_rating, 0) AS average_rating,
        COALESCE(c.comment_count, 0) AS number_of_comments
      FROM movies m
      LEFT JOIN (
        SELECT movie_id,
              COUNT(*) AS rating_count,
              ROUND(AVG(rating), 1) AS average_rating
        FROM ratings
        GROUP BY movie_id
      ) r ON m.id = r.movie_id
      LEFT JOIN (
        SELECT movie_id,
              COUNT(*) AS comment_count
        FROM comments
        GROUP BY movie_id
      ) c ON m.id = c.movie_id
      ORDER BY r.average_rating DESC, r.rating_count DESC
      LIMIT ?`,
      [limit]
    );
    return movies;
  }
```


getMovieDetails

This function fetches detailed information about a specific movie including comments, ratings, and whether the user has favorited it.

Parameters

- **movieId** (number): *Required.* The ID of the movie to retrieve details for.
- **userId** (number): *Required.* The ID of the user to check for favorites and ratings.

Returns

An object containing movie details, user rating, comments, and whether the user has favorited the movie.

Code

```
async function getMovieDetails(movieId, userId) {
  try {
    const [movieDetails] = await db.query(
      `SELECT
        m.*,
        COALESCE(r.rating_count, 0) AS number_of_raters,
        COALESCE(r.average_rating, 0) AS average_rating,
        COALESCE(c.comment_count, 0) AS number_of_comments,
        COALESCE(c.comments, '') AS comments,
        COALESCE(c.commenters, '') AS commenters,
        COALESCE(c.comment_times, '') AS comment_times
      FROM movies m
      LEFT JOIN (
        SELECT movie_id,
              COUNT(*) AS rating_count,
              ROUND(AVG(rating), 1) AS average_rating
        FROM ratings
        GROUP BY movie_id
      ) r ON m.id = r.movie_id
      LEFT JOIN (
        SELECT c.movie_id,
              COUNT(*) AS comment_count,
              COALESCE(GROUP_CONCAT(c.comment SEPARATOR " || "), "") AS comments,
              COALESCE(GROUP_CONCAT(u.name SEPARATOR " || "), "") AS commenters,
              COALESCE(GROUP_CONCAT(c.createdAt SEPARATOR " || "), "") AS comment_times
        FROM comments c
        LEFT JOIN users u ON c.user_id = u.id
        GROUP BY c.movie_id
      ) c ON m.id = c.movie_id
      WHERE m.id = ?;`,
      [movieId]
    );
    if (!movieDetails || movieDetails.length === 0) {
      return null;
    }
    const [userRatingResult] = await db.query(
      `SELECT rating FROM ratings WHERE movie_id = ? AND user_id = ?`,
      [movieId, userId]
    );
  }
}
```



```

const [userFavorited] = await db.query(
  `SELECT * FROM favorites WHERE user_id = ? AND movie_id = ?`,
  [userId, movieId]
);

return {
  movie: movieDetails[0],
  comments: movieDetails[0].comments
    ? movieDetails[0].comments.split(" || ")
    : [],
  commenters: movieDetails[0].commenters
    ? movieDetails[0].commenters.split(" || ")
    : [],
  comment_times: movieDetails[0].comment_times
    ? movieDetails[0].comment_times.split(" || ")
    : [],
  userRating: userRatingResult.length ? userRatingResult[0].rating : 0,
  userHasFavorited: userFavorited.length > 0,
}

```

getFavoriteMovies

This function retrieves all movies that a specific user has marked as favorites.

Parameters

- **userId** (number): *Required.* The ID of the user for whom to fetch favorite movies.

Returns

An array of favorite movie objects for the specified user.

Code

```

async function getFavoriteMovies(userId) {
  try {
    const [movies] = await db.query(
      `SELECT m.*,
        COALESCE(r.rating_count, 0) AS number_of_raters,
        COALESCE(r.average_rating, 0) AS average_rating,
        COALESCE(c.comment_count, 0) AS number_of_comments
      FROM movies m
      INNER JOIN favorites f ON m.id = f.movie_id
      LEFT JOIN (
        SELECT movie_id,
          COUNT(*) AS rating_count,
          ROUND(AVG(rating), 1) AS average_rating
        FROM ratings
        GROUP BY movie_id
      ) r ON m.id = r.movie_id
      LEFT JOIN (
        SELECT movie_id,
          COUNT(*) AS comment_count
        FROM comments

```

```

GROUP BY movie_id
    ) c ON m.id = c.movie_id
WHERE f.user_id = ?
ORDER BY m.id`,
[userId]
);

return movies

```

getTopMoviesPerCategory

This function gets the top-rated movies from each category, limiting the results to the top five per category.

Returns

An object where each key is a category and the value is an array of top movies in that category.

Code

```

async function getTopMoviesPerCategory() {
  try {
    const [movies] = await db.query(
      `SELECT movie_id, title, category, avg_rating
      FROM (
        SELECT
          m.id AS movie_id,
          m.title,
          m.category,
          COALESCE(AVG(r.rating), 0) AS avg_rating,
          ROW_NUMBER() OVER (PARTITION BY m.category ORDER BY COALESCE(AVG(r.rating), 0) DESC) AS rank_p
        FROM movies m
        LEFT JOIN ratings r ON m.id = r.movie_id
        GROUP BY m.id, m.title, m.category
      ) ranked_movies
      WHERE rank_position <= 5;`
    );

    if (!Array.isArray(movies) || movies.length === 0) {
      console.warn("No movies found in the database.");
      return {};
    }

    const groupedMovies = movies.reduce((acc, movie) => {
      if (!acc[movie.category]) {
        acc[movie.category] = [];
      }
      acc[movie.category].push(movie);
      return acc;
    }, {});

    return groupedMovies;
  }
}

```

getAllMovies

This function retrieves all movies from the database, including their ratings and comment information.

Returns

An array of all movie objects available in the database with their related statistics.

Example Usage

```
async function getAllMovies() {
  try {
    const [movies] = await db.query(`SELECT
      m.*,
      COALESCE(r.rating_count, 0) AS number_of_raters,
      COALESCE(r.average_rating, 0) AS average_rating,
      COALESCE(c.comment_count, 0) AS number_of_comments
    FROM movies m
    LEFT JOIN (
      SELECT movie_id,
        COUNT(*) AS rating_count,
        ROUND(AVG(rating), 1) AS average_rating
      FROM ratings
      GROUP BY movie_id
    ) r ON m.id = r.movie_id
    LEFT JOIN (
      SELECT movie_id,
        COUNT(*) AS comment_count
      FROM comments
      GROUP BY movie_id
    ) c ON m.id = c.movie_id
    ORDER BY r.average_rating DESC, r.rating_count DESC;`);

    return movies;
  }
}
```

Error Handling

In all functions, errors during database queries are caught and logged to the console. If an error occurs, a consistent return value (empty array or null) is used to ensure that the application can maintain predictable behavior in case of exceptions.

Exported Module

The following functions are exported from this module, making them available for import in other parts of the application:

```
module.exports = {
  getTopTenMovies,
  getMovieDetails,
  getFavoriteMovies,
  getTopMoviesPerCategory,
  getAllMovies,
};
```

FrontEnd JS Files Documentation

register.js

This JavaScript file is a key part of the MovieReviews project, responsible for handling the registration process of new users. It validates user input, communicates with the server, and provides user feedback during the registration process.

Overview

The `register.js` file listens for the submission of a registration form and takes care of all the necessary steps to register a new user. This includes gathering data from the form, validating the input, sending the data to the server, and handling the server's response.

Functionality Breakdown

Event Listener for DOMContentLoaded

At the start, an event listener waits for the document to be fully loaded. This ensures that all elements in the DOM are accessible before any scripts run.

```
document.addEventListener('DOMContentLoaded', () => { ... });
```

Sign Form Submission

The next task is to listen for the submission of the sign-up form. When the form is submitted, the default action (which would refresh the page) is prevented.

```
document.getElementById('sign-form').addEventListener('submit', async function (event) { event.preventDefault();
```

Data Gathering

Data is collected from the form inputs: username, email, password, and confirmation of the password. Each entry is trimmed to remove unnecessary whitespace.

```
const username = document.getElementById('username').value.trim(); const email = document.getElementById('em
const password = document.getElementById('password').value.trim(); const confirmPassword = document.getEleme
```

Redirect URL Handling

A redirect URL is determined from the current URL parameters. If none is provided, it defaults to `/home`.

```
const urlParams = new URLSearchParams(window.location.search); const redirect = urlParams.get('redirect') ||
```

Form Validation

Before proceeding, the code checks to ensure that all fields are filled. If any field is missing, an alert is displayed and submission is halted.

```
if (!username || !email || !password || !confirmPassword) { alert('Please fill in all fields.');
```


API Call for Registration

Taking the gathered data, it sends a POST request to the server to register the new user. The request contains the data in JSON format.

```
const response = await fetch('/api/auth/register', { method: 'POST', headers: { 'Content-Type': 'application',
,body: JSON.stringify({ username, email, password, confirmPassword, redirect })
```

Handling Server Response

The response from the server is awaited, and based on its status, appropriate actions are taken. If registration is successful, the user is redirected to the specified location. If it fails, an error message is displayed.

```
if (response.ok) { message.innerHTML = data.message; window.location.href = data.redirect; } else { message.
```

Error Handling

The code includes a try-catch block to gracefully handle any errors that may occur during the fetch process.

```
} catch (error) { alert(`Something went wrong. Please try again later.`); }
```


login.js

This file contains the JavaScript code responsible for handling the login functionality of the MovieReviews project. The code listens for the submission of a login form, validates the input, sends authentication requests, and provides feedback based on the server's response.

Event Listeners

The code begins by adding an event listener that waits for the entire HTML document to be fully loaded. This is essential to ensure that all elements are available to be manipulated.

```
document.addEventListener('DOMContentLoaded', () => {...
```

Once the document is ready, it sets up another event listener specifically for the login form.

Login Form Submission

The login form is targeted using its ID, and an event listener is added to handle the 'submit' event. Within this listener, we prevent the default form submission behavior to handle it with JavaScript.

```
document.getElementById('login-form').addEventListener('submit', async function (event) {...
```

The event is prevented from executing the default submission using `event.preventDefault()`, allowing custom processing.

Input Handling

The code retrieves the values of the email and password input fields, ensuring any leading or trailing whitespace is removed:

```
const email = document.getElementById('email').value.trim();  
const password = document.getElementById('password').value.trim();
```

It also prepares a message element where feedback will be displayed to the user:

```
const message = document.getElementById('message');
```

Redirect Handling

The code checks for any redirect URL parameters that might direct the user after a successful login. If no redirect is specified, it defaults to the home page:

```
const urlParams = new URLSearchParams(window.location.search);  
const redirect = urlParams.get('redirect') || '/home';
```

Validation

Before sending the request, the code checks if both the email and password fields have been filled in. If either is empty, an alert is shown:

```
if (!email || !password) {  
  alert('Please fill in both fields.');
```

API Request

The script then attempts to authenticate the user by sending a POST request to the server. This request is made using the `fetch` API and includes the email, password, and redirect URL:

```
const response = await fetch('/api/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password, redirect }));
```

Handling the Response

After awaiting the response, the server's reply is parsed as JSON:

```
const data = await response.json();
```

If the request is successful (`response.ok` is true), the user is redirected to the specified location:

```
if (response.ok) { ... }
```

Otherwise, an error message is shown to inform the user of what went wrong, using a nice icon for emphasis:

```
message.innerHTML = ` ${data.message}`;
```

Error Handling

Finally, if there is any error during the process (e.g., network issues), an alert informs the user that something went wrong:

```
alert('Something went wrong. Please try again later.');
```

home.js

This JavaScript file is part of the MovieReviews project. Its main purpose is to handle user sessions on the homepage of the application.

Overview

The `home.js` file ensures that users are informed if their session has expired. When the token is expired the user will redirect to `home.js`, the script checks for session status indicated by a URL parameter. If the session has expired, it alerts the user and redirects them to the login page.

Code Breakdown

Event Listener

The script starts with an event listener that waits for the DOM (Document Object Model) content to be fully loaded before executing its logic. This ensures that the HTML elements are available for manipulation.

Here's the relevant code snippet:

```
document.addEventListener('DOMContentLoaded', () => {
```

URL Parameter Handling

Next, the code retrieves URL parameters using the `URLSearchParams` object. This allows the script to fetch any query strings from the URL, particularly looking for a parameter named `expired`.

Here's how this is done:

```
const urlParams = new URLSearchParams(window.location.search);
```

Session Expiry Check

After extracting the URL parameters, the script checks if the `expired` parameter is set to `true`. If it is, the user is alerted about the session expiration and is redirected to the login page.

This logic is implemented as follows:

```
if (urlParams.get('expired') === 'true') {  
  alert('Your session has expired. Please log in again.');
```

```
  window.location.href = '/login';  
}
```

Navbar.js

This document provides a comprehensive overview of the `navbar.js` file in the Movie Reviews project. Its primary responsibility is to manage the search functionality for movies, displaying results dynamically based on user input.

Code Overview

The `navbar.js` file sets up an event listener for the document to handle user input in a search bar. It implements a search feature that, when triggered, fetches and displays movie results based on a user's query.

Event Listeners

The script listens for the `DOMContentLoaded` event to ensure the DOM is fully loaded before executing any code. This guarantees that all HTML elements are accessible for manipulation.

```
document.addEventListener('DOMContentLoaded', () => {  
  });
```

Search Input Initialization

Two key variables are initialized to store references to the input field and the results container:

```
const searchInput = document.getElementById('search');  
const resultsContainer = document.getElementById('searchResults');
```

Here, `searchInput` refers to the input field where users type their queries, and `resultsContainer` is where the search results will be displayed.

Debounce Mechanism

The code uses a debounce technique to optimize the search experience and reduce the number of API calls. It does this by introducing a delay before triggering the search until the user stops typing:

```
let debounceTimer;  
let abortController = new AbortController();  
  
searchInput.addEventListener('input', () => {  
  clearTimeout(debounceTimer);  
  
  const query = searchInput.value.trim();  
  
  if (query.length === 0) {  
    resultsContainer.innerHTML = '';  
    resultsContainer.classList.add('hidden');  
    return;  
  }  
  
  if (query.length < 2) return;
```



```

resultsContainer.innerHTML = `
  <div class="flex items-center">
    <p class="text-gray-500 py-4 pl-4">Searching...</p>
    <div class="w-6 h-6 border-4 border-yellow-400 border-t-transparent rounded-full animate-spin">
</div>
  </div>`;
resultsContainer.classList.remove('hidden');

debounceTimer = setTimeout(() => {
  fetchMovies(query);
}, 300);
});

```

Here, when the user types in the search input, any existing timeout is cleared and a new timeout is set. If the user stops typing, the `fetchMovies` function will be called after a 300ms delay.

Fetching Movies

Once the debounce time has passed, if the query length is valid (at least 2 characters), the `fetchMovies` function is called, which handles the API request:

```

async function fetchMovies(query) {
  abortController.abort();
  abortController = new AbortController();
  try {
    const response = await fetch(`/search?q=${query}`, {
      signal: abortController.signal,
    });
    const movies = await response.json();

    displaySearchResults(movies);
  } catch (error) {
    if (error.name !== 'AbortError') {
      console.error('Search Failed', error);
    }
  }
}

```

This function sends a fetch request to the server to retrieve movies that match the inputted query. The use of `AbortController` allows for cancelling ongoing fetch requests, thus avoiding unwanted responses from prior queries.

Displaying Results

The `displaySearchResults` function is responsible for rendering the list of movies returned from the fetch request:

```

function displaySearchResults(movies) {
  resultsContainer.innerHTML = '';

  if (movies.length === 0) {
    resultsContainer.innerHTML =
      '<p class="text-gray-600 p-4">No Movies Found</p>';
    return;
  }
}

```


This function first clears any existing results in the `resultsContainer`. If no movies are found, it will display a friendly message saying *No Movies Found*. Otherwise, it loops through the movie array and constructs the HTML elements to display each movie's details such as title, category, and duration.

Example of Result Rendering

Here is an example of how each movie is displayed:

```
movies.forEach((movie) => {
  const hour = Math.floor(movie.duration / 60);
  const remainingMinutes = movie.duration % 60;
  const movieItem = document.createElement('div');
  movieItem.className = 'p-2 border-b';
  movieItem.innerHTML = `
    <a href="/${a}/${movie.id}" class="flex items-center gap-4 p-3 rounded-lg transition duration-200
    hover:bg-gray-200">
      
      <div class="flex flex-col">
        <p class="text-lg font-semibold text-gray-900 ">${movie.title}</p>
        <p class="text-base text-gray-800">${movie.category}</p>
        <div class="flex items-center gap-2 text-gray-800">
          <p class="text-sm">${movie.release_year}</p>/
          <p class="text-sm">${hour}h ${remainingMinutes}m</p>
        </div>
      </div>
    </a>
  `;
  resultsContainer.appendChild(movieItem);
}
```

This code creates a `div` element for each movie that includes an anchor tag linking to either a review page or a guest review page based on the user status.

guestNav.js

This JavaScript file enhances the user experience on the guest navigation bar of the Movie Reviews project. It ensures that users are redirected appropriately when they try to log in or register from certain pages.

File Overview

The **guestNav.js** script handles the functionality associated with the guest navigation links (login and signup) on a web page. It dynamically generates the redirect URLs for users once they log in or sign up, based on their current page.

Code Explanation

Event Listener for DOMContentLoaded

The first line of the code registers an event listener that waits for the HTML document to be fully loaded before executing the supplied function. This is crucial to ensure all the elements in the document are accessible and ready to be manipulated.

```
document.addEventListener('DOMContentLoaded', () => { ... });
```

Getting Elements by ID

Two variables, `loginLink` and `signupLink`, are initialized. They retrieve anchor (link) elements from the HTML document using their respective IDs:

```
const loginLink = document.getElementById('loginLink');
const signupLink = document.getElementById('signupLink');
```

Current Path Retrieval

The current URL path is stored in the `currentPath` variable by accessing the `window.location.pathname`. This helps in determining where the user is currently navigating from:

```
const currentPath = window.location.pathname;
```

Redirect Path Logic

A new variable, `redirectPath`, is created. It computes the appropriate redirect URL based on `currentPath`:

- If the user is on a guest review page, it replaces `"/guestReview/"` with `"/review/"`.
- If the user is on the main page `'/'`, they will be redirected to `"/home"`.
- Otherwise, it simply uses the current path.

```
const redirectPath = currentPath.startsWith('/guestReview/')
  ? currentPath.replace('/guestReview/', '/review/')
  : currentPath === '/'
  ? '/home'
  : currentPath
```

Encoding the Redirect Path

The next line encodes the `redirectPath` to ensure it is safe to include in a URL:

```
const encodedRedirect = encodeURIComponent(redirectPath);
```

Setting the Login and Signup Links

Finally, the `href` attributes of the login and signup links are set to include the encoded redirect path. This ensures that once a user logs in or signs up, they will be redirected correctly:

```
loginLink.href = `/login?redirect=${encodedRedirect}`;  
signupLink.href = `/register?redirect=${encodedRedirect}`;
```

categories.js

This code file is responsible for fetching and rendering movie categories in the MovieReviews web application. It allows users to view movies organized by categories and handles displaying these movies differently based on the user's login state.

Overview of Main Functionality

The script executes a series of functions to fetch movie data from a designated server endpoint. Based on the user's authentication status, it then renders the relevant movie categories into the DOM. Additionally, it provides a toggle feature for a menu overlay where users can access the movie categories.

Event Listener for DOMContent Loaded

The code begins by waiting for the DOM to be fully loaded before executing the enclosed functions. This ensures that the HTML elements are accessible when the script runs.

The initial part of the code looks like this:

```
document.addEventListener('DOMContentLoaded', () => { ... });
```

fetchMovies Function

This asynchronous function is responsible for fetching movie data from the endpoint `</menu/categories>`. It processes the response to retrieve the filtered movies and the user's login status, then calls the `renderMovies` function to display them.

Here's how the code looks:

```
async function fetchMovies() {
  try {
    const response = await fetch('/menu/categories');
    if (!response.ok) throw new Error('Failed to fetch movies');

    const { filteredMovies, userIsLoggedIn } = await response.json();
    renderMovies(filteredMovies, userIsLoggedIn);
  } catch (error) {
    console.error('Error fetching movies:', error);
    document.getElementById('movies-container').innerHTML =
      'Failed to load movies.';
  }
}
```

renderMovies Function

This function dynamically creates and populates the movie categories in the HTML using a loop to iterate over the categories of films. It generates HTML elements and appends them to the movie container.

The function is structured as follows:

```
function renderMovies(filteredMovies, userIsLoggedIn) {
  const moviesContainer = document.getElementById('movies-container');
  moviesContainer.innerHTML = '';
```


Each category is represented with a header and a list of movies linking to their respective review pages, which changes based on the user's login status:

```
for (const category in filteredMovies) {
  const categorySection = document.createElement('div');
  categorySection.innerHTML = `...`;
  moviesContainer.appendChild(categorySection);
}
```

```
${movie.title}
```

toggleMenu Function

This function shows or hides a menu overlay. It alters the overlay's visibility and applies or removes CSS animations for a smoother user experience. This helps manage the flow of navigation.

Here's the basic structure of the `toggleMenu` function:

```
function toggleMenu() {
  const menuOverlay = document.getElementById('menu-overlay');
  const menuContent = document.getElementById('menu-overlay1');
  const body = document.body;
  body.classList.toggle('overflow-hidden');

  if (menuOverlay.classList.contains('hidden')) {
    menuOverlay.classList.remove('hidden');
    menuContent.classList.remove('animate-slide-out');
    menuContent.classList.add('animate-slide-in');
    fetchMovies();
  } else {
    menuContent.classList.remove('animate-slide-in');
    menuContent.classList.add('animate-slide-out');

    setTimeout(() => {
      menuOverlay.classList.add('hidden');
    }, 700);
  }
}
```

Menu Button Event Listener

Lastly, the script attaches click event listeners to the menu buttons. If either of the buttons is clicked, the `toggleMenu` function will be invoked to either display or hide the movie categories menu.

```
const menuButton = document.getElementById('menu-toggle-btn');
const menuButton2 = document.getElementById('menu-toggle-btn2');

if (menuButton) menuButton.addEventListener('click', toggleMenu);
if (menuButton2) menuButton2.addEventListener('click', toggleMenu);
```


review.js

This script is responsible for handling user interactions related to movie ratings ,favorite and comments.

Overview

This JavaScript file is designed to manage user interactions on the movie review page, allowing users to rate movies, post comments, and mark movies as favorites. It uses event listeners to gather user input and communicate with the backend via fetch API calls.

Key Variables

Here's a quick run-down of some important variables:

- `stars` : Selects all the star icons used for ratings.
- `ratingStars` : Reference to the rating display area.
- `ratingResult` : Element that shows the average rating of the movie.
- `numberOfRaters` : Displays how many users have rated the movie.
- `favoriteIcon` : An icon allowing users to mark the movie as a favorite.
- `submitButton` : The button that users click to submit their comment.
- `commentInput` : The input field where users type their comments.
- `currentRating` : Holds the user's current rating.
- `tempRating` : A temporary variable to hold rating while the user hovers over stars.
- `isFavorited` : A boolean that checks if the movie is marked as a favorite by the user.

Event Listener: DOMContentLoaded

The script begins by listening for the `DOMContentLoaded` event to ensure the HTML is fully loaded before executing the script. Here's how it starts:

```
document.addEventListener('DOMContentLoaded', () => {
```

This way, we can safely access elements in the document, like stars and buttons!

Check User Status

At the first this script check if the user logged in or not and act per that , if not it will disable all of the actions.

```
if (userIsLoggedIn === 'false') {  
  if (ratingStars) {  
    ratingStars.querySelectorAll('.fa-star').forEach((star) => {  
      star.classList.add('opacity-50', 'cursor-not-allowed');  
    });  
  }  
}
```

```
commentInput.disabled = true;
commentInput.placeholder = 'Please log in to add a comment';
submitButton.classList.add('opacity-50', 'cursor-not-allowed');
submitButton.disabled = true;
favoriteIcon.classList.add('opacity-50', 'cursor-not-allowed');
```

And if the user logged in it will accept the reviews.

Star Rating Functionality

Among the stars, the `updateStars(rating)` function updates the stars based on the specified rating. Here is how it works:

```
function updateStars(rating) {
  stars.forEach((star) => {
    if (star.getAttribute('data-rating') <= rating) {
      star.classList.add('text-yellow-500');
    } else {
      star.classList.remove('text-yellow-500');
    }
  });
}
```

This function iterates through each star and highlights it based on the user's current rating.

The stars also have event listeners for `mouseenter`, `mouseleave`, and `click` events, allowing users to interactively select their rating. For example, on click, it sends the rating to the server:

```
star.addEventListener('click', async () => {
  currentRating = star.getAttribute('data-rating');
  updateStars(currentRating);

  const response = await fetch(`/movies/${movieId}/rate`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ rating: currentRating }),
  });
});
```

Comment Submission Functionality

User comments are submitted through another event listener on the `submitButton`. Here's how it handles the click event:

```
submitButton.addEventListener('click', async () => {
  const comment = commentInput.value.trim();
  if (!comment) return;

  const response = await fetch(`/movies/${movieId}/comment`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ comment }),
  });
});
```

If the comment submission is successful, the new comment will be displayed at the top of the comments section.

Favorite Functionality

The script also allows users to favorite a movie. When the `favoriteIcon` is clicked, it sends a request to the server to toggle the favorite status:

```
favoriteIcon.addEventListener('click', async () => {  
  const response = await fetch(`/movies/${movieId}/favorite`, {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
  });  
});
```

Depending on the response from the server, the icon will update to reflect the new favorite status!

Dashboard.js

This JavaScript file is responsible for managing the dashboard functionalities of the Movie Reviews project. It provides features for opening and closing movie details modals, adding new movies, updating existing movies, deleting movies, and filtering user and movie data. With this file, admins can efficiently manage movies and user roles from the dashboard.

Initialization

The script initiates once the HTML document's content is fully loaded. This ensures that all elements are ready for manipulation.

```
document.addEventListener('DOMContentLoaded', () => { ... });
```

Modal Management

The modal for adding new movies is handled with functions to open and close the modal, allowing users to input information.

Open Modal

This function removes the 'hidden' class from the modal, making it visible.

```
function openModal() { movieModal.classList.remove('hidden'); }
```

Close Modal

This function adds the 'hidden' class back to the modal and resets the movie form fields.

```
function closeModal() { movieModal.classList.add('hidden'); movieForm.reset(); }
```

Handling Movie Submission

When the form is submitted, the movie data is collected and sent to the server to be added to the database. The user is notified upon success or failure.

```
movieForm.addEventListener('submit', async (event) => {
  event.preventDefault();
  const movieData = {
    title: document.getElementById('movieTitle').value.trim(),
    description: document.getElementById('movieDescription').value.trim(),
    release_year: document.getElementById('movieReleaseYear').value.trim(),
    image_url: document.getElementById('movieImageUrl').value.trim(),
    producer: document.getElementById('movieProducer').value.trim(),
    category: document.getElementById('movieCategory').value.trim(),
    duration: document.getElementById('movieDuration').value.trim(),
  };
  try {
    const response = await fetch('/admin/addMovies', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(movieData),
    });
  }
```


This event handler submits the form asynchronously so the page does not refresh. It uses the Fetch API to send a POST request with movie data.

Update and Delete Movies

Functions to update and delete movies are provided through asynchronous requests to the server.

Update Movie

This function collects updated data from the respective movie row and sends a PATCH request.

```
async function updateMovie(movieRow) {
  const movieId = movieRow.dataset.movieId;
  const updatedData = {
    title: movieRow.querySelector("input[type='text']").value.trim(),
    description: movieRow
      .querySelectorAll("input[type='text']")[1]
      .value.trim(),
    release_year: parseInt(
      movieRow.querySelector("input[type='number']").value.trim(),
      10
    ),
    producer: movieRow.querySelectorAll("input[type='text']")[2].value.trim(),
    image_url: movieRow.querySelector("input[type='url']").value.trim(),
    category: movieRow.querySelector('select').value.trim(),
    duration: parseInt(
      movieRow.querySelectorAll("input[type='number']")[1].value.trim(),
      10
    ),
  };
  try {
    const response = await fetch(`/admin/updateMovie/${movieId}`, {
      method: 'PATCH',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(updatedData),
    });
  }
```

Delete Movie

Prompts the user for confirmation before sending a DELETE request for a specified movie.

```
async function deleteMovie(movieRow) {
  const movieId = movieRow.dataset.movieId;
  if (!confirm('Are you sure you want to delete this movie?')) return;
  try {
    const response = await fetch(`/admin/deleteMovie/${movieId}`, {
      method: 'DELETE',
    });
  };
  const result = await response.json();
  if (response.ok) {
    alert('Movie deleted successfully!');
    movieRow.remove();
  }
```


Event Delegation for Movie Table

Event listeners are attached to the movie table to handle updates and deletions based on user actions.

```
movieTable.addEventListener('click', (event) => {
  const target = event.target;
  const movieRow = target.closest('tr');

  if (!movieRow) return;

  if (target.classList.contains('save-movie')) {
    updateMovie(movieRow);
  } else if (target.classList.contains('delete-movie')) {
    deleteMovie(movieRow);
  }
})
```

Within this handler, the target of the click event is checked to determine if it relates to saving or deleting a movie.

User Management

Similar functionalities are provided for managing users; adding admin roles and deleting users can be handled directly from the user table.

Make Admin

Updates a user's role to admin through a PATCH request.

```
async function makeAdmin(userId) {
  try {
    const response = await fetch(`/admin/userRole/${userId}`, {
      method: 'PATCH',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ role: 'admin' }),
    });
  }
```

Delete User

Prompts the user for confirmation before sending a DELETE request for the specified user.

```
async function deleteUser(userId) {
  if (!confirm('Are you sure you want to delete this user?')) return;

  try {
    const response = await fetch(`/admin/deleteUser/${userId}`, {
      method: 'DELETE',
    });
  }
```

Filtering Functionality

Input fields for filtering users and movies are hooked up to their respective rows to provide a dynamic search experience.

Filter Users

Filters user data in the table based on email input.

```
emailFilterInput.addEventListener('input', function () {
  const filterValue = emailFilterInput.value.trim().toLowerCase();
  const rows = userTableBody.getElementsByTagName('tr');

  for (let row of rows) {
    const emailCell = row.getElementsByTagName('td')[2];
    if (emailCell) {
      const emailText = emailCell.textContent.toLowerCase();
      row.style.display = emailText.includes(filterValue) ? '' : 'none';
    }
  }
});
```

Filter Movies

Filters movie data based on title input, maintaining a smooth user experience.

```
titleFilterInput.addEventListener('input', function () {
  const filterValue = titleFilterInput.value.trim().toLowerCase();
  const rows = movieTableBody.getElementsByTagName('tr');

  for (let row of rows) {
    const titleCell = row.getElementsByTagName('td')[1];
    if (titleCell) {
      const titleText = titleCell.querySelector('input').value.toLowerCase();
      row.style.display = titleText.includes(filterValue) ? '' : 'none';
    }
  }
});
```

This document serves as a comprehensive guide to the **dashboard.js** file in the Movie Reviews project, detailing all essential functions and their respective interactions.